

Лекция 3.2

- Верификация и валидация требований



Верификация и валидация требований

Стандарт ИСО 9000:2000 определяет эти термины следующим образом:

«**Верификация** — подтверждение на основе представления объективных свидетельств того, что установленные требования были выполнены».

«**Валидация** — подтверждение на основе представления объективных свидетельств того, что требования, предназначенные для конкретного использования или применения, выполнены».

Как видно, определения чуть ли не совпадают и уж если не полностью, то в значительной части. И, тем не менее, **верификация и валидация — принципиально разные действия.**

Перевод с английского этих терминов позволяет понять разницу:

verification — проверка,

validation — придание законной силы.



Верификация требований

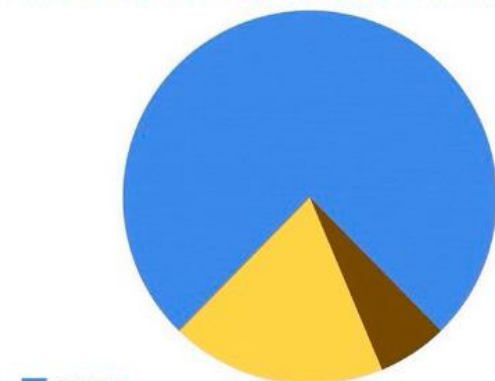
Верификация требований – это проверка, что их описание в виде спецификации (SRS) или ТЗ соответствуют отраслевым и организационным стандартам качества, а также пригодны для дальнейшего практического использования. Последнее означает, *что на основании составленных аналитиком требований ИТ-архитектор сможет выполнить архитектурный проект, разработчик – написать код, UI/UX-дизайнер – реализовать пользовательский интерфейс, а тестировщик – протестировать, как все это работает.*

Способы верифицировать требование:

- ✓ проверка требования на соответствие критериям качества.
- ✓ проверка правильности описания бизнес-процессов в формальных нотациях моделирования (IDEF0, BPMN, EPC или UML).
- ✓ проверка, насколько разумно использовать выбранные методы и инструменты с учетом дальнейшего применения этих диаграмм и способности стейкхолдеров их понимать.

Верификация требований – это не однократное мероприятие, а целый набор действий, которые аналитик периодически выполняет в рамках разработки и анализа требований.

КАК Я ВИЖУ ЛЮБЫЕ ДИАГРАММЫ



это шутка

Валидация требований

Валидация требований – это тоже проверка. Однако, в отличие от верификации, она направлена не на форму, а на содержание. Цель валидации требований – убедиться, что все требования стейкхолдеров и требования к решению соответствуют бизнес-требованиям и смогут удовлетворить бизнес-потребности.

Этот непрерывный процесс включает следующие действия:

- ✓ **выявление предположений**, что реализация требования поможет бизнесу и стейкхолдерам получить ожидаемую ценность. Сформулировав такие предположения, аналитик и стейкхолдеры подтверждают или опровергают их.
- ✓ **определение измеримых критериев оценки**, целевых показателей и метрик, а также их значений, чтобы понять успех изменения после реализации решения, т.е. после перехода от текущего состояния (as is) к желаемому (as to be).
- ✓ **оценка соответствия границам и содержанию решения** – входит ли требование в *scope*. Хотя требование может быть полезно конкретному стейкхолдеру, оно может противоречить бизнес-требованиям и/или выходить за рамки решения. В этом случае следует пересмотреть будущее состояние и изменить *scope* решения или исключить требование. Если вариант решения нельзя проверить на соответствие требованию, то оно отсутствует или неправильно понято.

Чем валидация отличается от верификации?

Верификация и валидация являются проверками, это две разных задачи из одной области знаний BABOK Guide «Анализ требований и определение дизайна» (Requirements Analysis and Design Definition).

Верификация — проводится практически всегда, выполняется методом проверки (сличения) характеристик продукции с заданными требованиями, **результатом является вывод о соответствии (или несоответствии) продукции.**

Валидация — проводится при необходимости, выполняется методом анализа заданных условий применения и оценки соответствия характеристик продукции этим требованиям, результатом является **вывод о возможности применения продукции для конкретных условий.**

Верификация отвечает на вопрос «Правильно ли записаны требования?», а валидация — на вопрос «А правильные ли требования записаны?».

или

Верификация отвечает на вопрос "Делаем ли мы продукт правильно?", а валидация- на вопрос "Делаем ли мы правильный продукт?"



Что не прошло это требование верификацию или валидацию?

Чем валидация отличается от верификации?

Верификация	Валидация
Делают ли разработчики продукт правильно	Правильный ли получился проект
Все ли функции реализованы	Насколько грамотно реализована функциональность
Предшествует валидации. Включает в себя полноценную проверку правильности написания.	Проводится после верификации. Это – оценка качества итогового проекта.
Испытания организовываются разработчиками	Испытания организованы тестировщиками
Тип анализа – статистический. Проводится сравнение с установленными требованиями к итоговому проекту.	Тип анализа – динамический. Проект проходит испытания по эксплуатации. Это помогает понять, насколько продукт соответствует действующим нормам.
Оценка объективна. Она базируется на соответствии определенным стандартам.	Оценка субъективна. Она является личной. Это – оценка, которую ставит каждый тестировщик.

Рецензирование требований

Всякое исследование продукта ПО на предмет выявления проблем любым другим лицом, кроме его автора, называется рецензированием (peer review).

Неформальное рецензирование полезно при знакомстве людей с продуктом и сборе неструктурированных отзывов о нем. Есть несколько видов:

- ✓ проверка «за столом» (peer deskcheck), когда вы просите одного коллегу исследовать ваш продукт;
- ✓ коллективная проверка (passaround), когда вы приглашаете несколько коллег для параллельной проверки продукта;
- ✓ сквозной разбор (walkthrough), когда автор описывает создаваемый продукт и просит его прокомментировать.

Формальное рецензирование представляет собой строго регламентированный процесс. По его завершении формируется отчет, в котором указаны материал, рецензенты и мнение команды рецензентов о приемлемости продукта.

Наиболее хорошо себя зарекомендовавшая себя форма официального рецензирования **экспертиза** (inspection). Экспертиза это четко определенный многоэтапный процесс. В ней участвует небольшая команда участников, которые тщательно проверяют продукт на наличие дефектов и изучают возможности улучшения.

Пример, несколько компаний сэкономили ни много ни мало — целых 10 часов труда на каждый час, потраченный на экспертизу документации требований и других готовых к поставке продуктов (Grady и Van Slack, 1994).

Процесс экспертизы

Участники экспертизы должны отражать четыре точки зрения на продукт (Wiegers):

- ✓ **Автор продукта или группа авторов.** Это точка зрения бизнесаналитика, составившего документацию требований.
- ✓ **Люди, послужившие источником информации для элемента, который следует проверять.** Это представители пользователей или автор предыдущей спецификации.
- ✓ **Люди, которые будут выполнять работу, основанную на проверяемом элементе** Можно привлечь разработчика, тестировщика, менеджера проекта, а также составителя пользовательской документации, потому что они смогут выявить проблемы различных типов. Тестировщик выявит требования, не поддающиеся проверке, а разработчик сумеет определить требования, которые технически невыполнимы.
- ✓ **Люди, отвечающие за работу продуктов, взаимодействующих с проверяемым элементом.** Они выявят проблемы с требованиями к внешнему интерфейсу. Они также могут выявить требования, изменение которых в проверяемой спецификации повлияет на другие системы (эффект волны).

Все участники экспертизы, включая автора, ищут недостатки и возможности улучшения.

Пример из книги Вегерса: в Chemical Tracking System представители пользователей проводили неофициальное рецензирование последней версии спецификации требований после каждого семинара, посвященного выявлению требований. Таким образом удавалось выявить множество ошибок. После завершения выявления требований, один аналитик свел всю информацию, полученную от всех классов пользователей в одну спецификацию требований к ПО — 50 страниц плюс несколько приложений. Затем **два бизнес-аналитика, один разработчик, три сторонника продукта, менеджер проекта и один тестировщик** обследовали этот документ за **три двухчасовых совещания**, проведенных в течение недели. Они дополнительно обнаружили **233 ошибки, в том числе десятки серьезных дефектов.**

Предотвращение неопределенности

Качество требований определяет читатель, а не автор. Бизнес-аналитик может считать, что написанное им требование кристально-ясное, свободно от неоднозначностей и других проблем. Но если у читателя возникают вопросы, требование нуждается в дополнительном совершенствовании. **Рецензирование — лучший способ поиска мест, где требования непонятны всем целевым аудиториям.**

Одной из причин возникновения неопределенности в требованиях является то, что требования пишутся на **естественном языке** (natural language), в отличие от **формального языка** (formal language), в котором двусмысленность исключена.

Формальный язык — это язык, который характеризуется точными правилами построения выражений и их понимания. К формальным языкам относятся формулы, блок-схемы, алгоритмы и т.д.

Естественный язык — это язык, который используется в повседневной жизни прежде всего для общения, и имеет целый ряд особенностей, к примеру, слова могут иметь более одного значения, иметь синонимы или быть омонимами, а иногда значения отдельных слов и выражений зависят не только от них самих, но и от контекста, в котором они употребляются.

Все это в совокупности приводит к тому, что **причиной возникновения неопределенности как раз и является естественный язык.**

Предотвращение неопределенности

Требования, изложенные неясным языком, не поддаются проверке, поэтому нужно избегать двусмысленных и субъективных терминов. В таблице (*табл. 11-2 стр. 250 в книге Разработка требований к программному обеспечению Карла Вигерса*) перечислены многие из них, а также приводятся рекомендации, как исправить такие неясности.

Ниже приведены отдельные части из этой таблицы:

Неоднозначные термины	Способы улучшения
Приемлемый, адекватный	Определите, что понимается под приемлемостью и как система это может оценить
И/или	Укажите точно, что имеется в виду — «и» или «или», чтобы не заставлять читателя гадать
Между, от X до Y	Укажите, входят ли конечные точки в диапазон
Зависит от	Определите природу зависимости. Обеспечивает ли другая система ввод данных в вашу систему, надо ли установить другое ПО до запуска вашей системы и зависит ли ваша система от другой при выполнении определенных расчетов или служб?

Неоднозначные термины	Способы улучшения
Эффективный	Определите, насколько эффективно система использует ресурсы, насколько быстро она выполняет определенные операции и как быстро пользователи с ее помощью могут выполнять определенные задачи
Быстрый, скорый, моментальный	Укажите минимальное приемлемое время, за которое система выполняет определенное действие
Гибкий, универсальный	Опишите способы адаптации системы в ответ на изменения условий работы, платформ или бизнес-потребностей
Улучшенный, лучший, более быстрый, превосходящий, более качественный	Определите количественно, насколько лучше или быстрее должны стать показатели в определенной функциональной области или аспект качества
Обычно, в идеальном варианте	Опишите нештатные или неидеальные условия и как система должна вести себя в таких ситуациях
Необязательно	Укажите, кто делает выбор: система, пользователь или разработчик
Возможно, желательно, должно	Должно или не должно?

Неоднозначные термины	Способы улучшения
Устойчивый к сбоям	Определите, как система должны обрабатывать исключения и реагировать на неожиданные условия работы
Цельный, прозрачный, корректный	Что означает «цельный» или «корректный» для пользователя? Выразите ожидания пользователя, применяя характеристики продукта, которые можно наблюдать
Несколько, некоторые, много, немного, множественный	Укажите сколько или задайте минимальную и максимальную границы диапазона
Не следует	Старайтесь формулировать требования в позитивной форме, описывая, что именно система будет делать
Современный	Поясните этот термин для заинтересованного лица
Достаточный	Укажите, какая степень чего-либо свидетельствует о достаточности
Поддерживает, позволяет	Дайте точное определение, из каких действий системы состоит «выполнение» конкретной возможности
Дружественный, простой, легкий	Опишите системные характеристики, которые будут отвечать потребностям пользователей и его ожиданиям, касающимся легкости и простоты использования продукта

Пограничные значения

Много неоднозначности возникает на границах числовых диапазонов как в требованиях, так и бизнес-правилах. Пример:

Отпуск длительностью до 5 дней не требует одобрения. Запросы на отпуск длительностью от 5 до 10 дней требуют одобрения непосредственного начальника. Отпуска длительностью 10 дней и более требуют одобрения директора.

При такой формулировке непонятно, в какую категорию попадают отпуска длительностью точно 5 и 10 дней. Слова «от и до», «включительно» и «свыше» вносят четкость и ясность насчет пограничных значений:

Отпуск длительностью 5 дней и меньше не требует одобрения. Запросы на отпуск длительностью более 5 и до 10 дней включительно требуют одобрения непосредственного начальника. Отпуска длительностью свыше 10 дней требуют одобрения директора.

Негативные требования

Иногда люди пишут в требованиях не то, система должна, а то, что она не должна делать. Как реализовать такие «негативные» требования? Особенно сложны в расшифровке двойные и тройные отрицания. Попробуйте переформулировать негативные требования в позитивном стиле, которые описывают ограничение на поведение. Вот пример:

Пользователь не должен иметь возможность активизировать договор, если он не сбалансирован.

Лучше перефразировать это двойное отрицание («не должен» и «не сбалансирован») как позитивное утверждение:

Система должна позволять пользователю активировать договор, только если этот договор сбалансирован

Пример двойного отрицания:

Be positive & avoid double negatives

Don't	Do
IF the User has typed-in at least one character into the field, THEN the system does NOT show the validation message.	IF the User has left the field blank , THEN the system shows the validation message.

Если пользователь ввел хотя бы один символ в поле, тогда система не показывает валидационное сообщение. Возникает вопрос, а что тогда должна желать система, если она не показывает? Правильно написать: Если пользователь оставил поле пустым, тогда система показывает валидационное сообщение

IF the Customer has selected NO Products, THEN the system does NOT allow to make the Order.	IF the Customer has selected at least one Product, THEN the system allows to make the Order.
--	---

Если пользователь не выбрал ни одного продукта, тогда система не позволяет сделать заказ.
Лучше: Если пользователь выделил хотя бы один продукт, тогда система позволяет сделать заказ.

Слова, которые приводят к двусмысленности

К словам, которые могут приводить к двусмысленности, относятся:

- соединительный союз «и»;
- разделительный союз «или»;
- пояснительные союзы «также» и «только»;
- слова «по возможности», «при необходимости».

Это далеко не весь список. Однако мне бы хотелось рассмотреть именно его, поскольку без этих слов иногда не обойтись, но применять их нужно очень осторожно.

Соединительный союз «и»

Этот союз может означать, что:

- одно действие или объект в хронологическом порядке следует за другим действием или объектом;
- одно действие является результатом другого действия;
- действие или объект зависит от предыдущего действия или объекта.

На одном из проектов аналитиком было прописано требование:ы «*Все пользователи для входа в систему используют логин и пароль*». При этом, подразумевалось, что «У каждого пользователя для входа в систему есть свой уникальный логин и пароль». Но при прочтении требований разработчик интерпретировал это требование как «Все пользователи используют один логин и пароль для входа в систему».

Пояснительные союзы «также» и «только»

Могут вносить неоднозначность в сочетании с «и» и «или», образуя сочетания:

только...или;

также...и.

Например, *«Только администратор и координатор могут заблокировать пользователя»*. Возможные трактовки данного требования:

- один администратор и любой координатор **вместе** могут заблокировать пользователя;
- каждый администратор может заблокировать пользователя;
- каждый координатор может заблокировать пользователя.

Слова «по возможности», «при необходимости»

Использование этих слов вносит не только двусмысленность в написанное, но еще делают требования не тестируемым, поскольку неясно, в какой момент эта «возможность» или «необходимость» должна появиться.

«При необходимости пользователь может распечатать счет». Данное требования не дает точной информации, когда должна быть доступна эта функция.

[Еще больше примеров: Как избежать двусмысленности в требованиях \(analyst.by\)](#)

Gherkin - человеко-читаемый язык для описания поведения системы. Каждая строчка начинается с одного из ключевых слов и описывает один из шагов. **Такая формализация помогает исключить неопределенности.**

Don't	Do
The system applies the Discount Code IF the Customer submits one. The system does it ONLY IF the Customer has been verified .	GIVEN the Customer views <i>New Order</i> , GIVEN the Customer has been verified , IF the Customer submits the Discount Code, THEN the system applies the Discount Code .
IF the Phone Number is new , THEN the system sends a Verification Message .	GIVEN the User has Notification Method set to " SMS ", IF the User updates their Profile AND saves a new Phone Number , THEN the system sends a Verification Message .